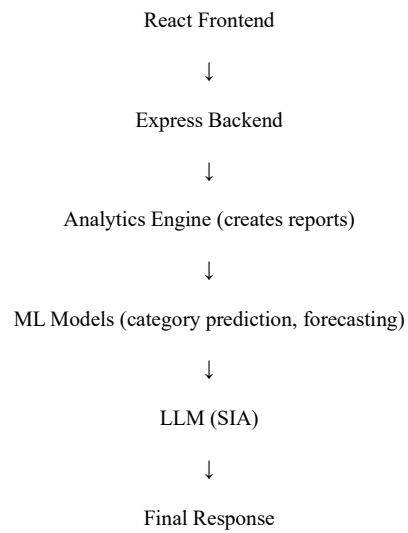


## Introducing **SIA** from the world of **BALENISA**

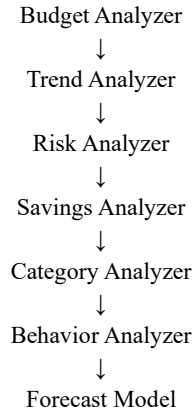
### Architecture:



### Create Analytics Models

Instead of one report,

Have many analyzers.



Each returns JSON.

Example

Budget Analyzer

```
{
  usage:82,
  remaining:1800,
  risk:"Medium"
}
```

## Trend Analyzer

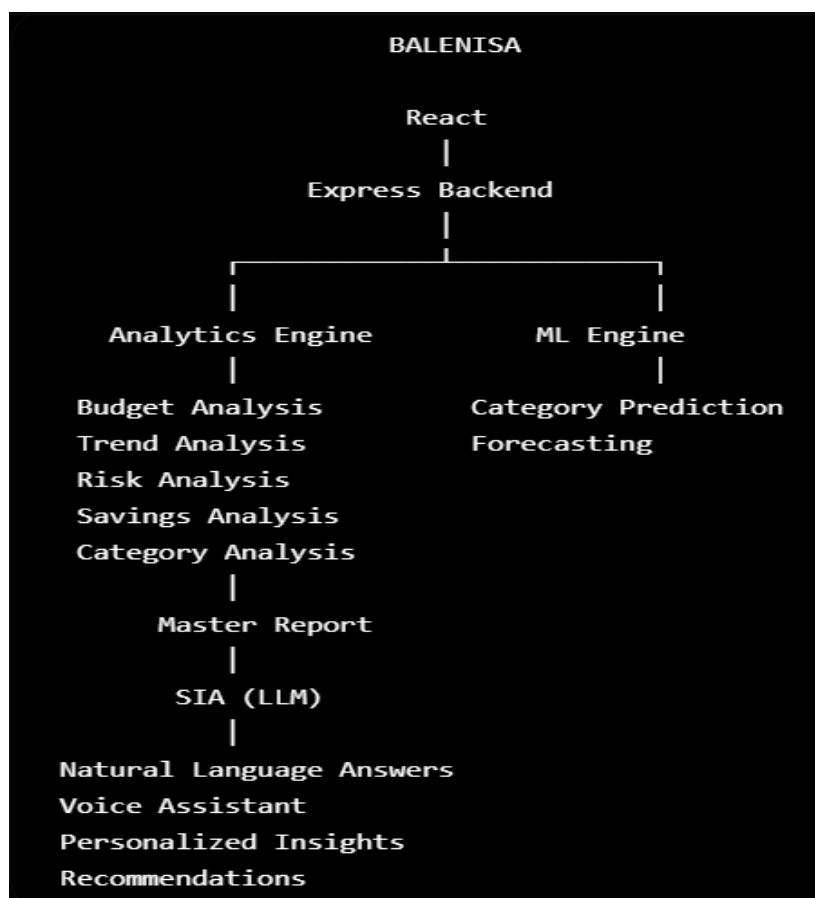
```
{  
  growth:18,  
  direction:"Increasing"  
}
```

## Behavior Analyzer

```
{  
  weekendRatio:40,  
  nightSpending:12,  
  largestCategory:"Food"  
}
```

Then combine.

Master Report  
↓  
LLM



## Design

### BALENISA Architecture

- └─ 1. Authentication Module
- └─ 2. Expense Management
- └─ 3. Budget Management
- └─ 4. Analytics Engine
- └─ 5. Machine Learning Engine
- └─ 6. SIA AI Assistant
- └─ 7. Dashboard
- └─ 8. Notifications
- └─ 9. Settings

### Module 1 — Authentication

#### Responsibilities

- Signup
- Login
- JWT
- Password hashing

#### Future

- Profile

#### ML?

✗ None

---

### Module 2 — Expense Management

This is your CRUD layer.

#### Features

- Add expense
- Edit expense
- Delete expense
- Recurring expenses
- Categories

## Future

- Search
- Filters (Amount Range Filters)
- Pagination (**Don't add visible pagination.** Keep the continuous scroll, but change your data loading so it fetches expenses in batches (e.g., 50 at a time).

## ML Used?

☒ Expense Categorization

This model already exists.

---

## Module 3 — Budget Module

### Features

- Monthly budgets
- Remaining budget
- Progress
- Analysis

## ML?

### Initially

☐ No

### Later

### Budget Risk Prediction

### Example

Current date

15th

Budget used

78%

Previous months show user usually spends more during weekends.

Prediction

92% probability of exceeding budget

Possible models:

- Random Forest
  - XGBoost
- 

## Module 4 — Analytics Engine

This is the most important module.

This module **does not use ML**.

It performs calculations.

Input

**Expenses**

**Income**

**Budgets**

Output **Financial Report**

Example

```
{  
  "monthlyExpense": 42000,  
  "monthlyIncome": 50000,  
  "savings": 8000,  
  "highestCategory": "Food",  
  "averageDailySpend": 1350,  
  "largestExpense": "...",  
  "weekendRatio": 34,  
  "budgetUsage": 82,  
  "monthlyGrowth": 18  
}
```

Everything is statistics.

No AI.

No LLM.

---

Analytics submodules

### Spending Analyzer

Computes

- Total expense

- Daily average
  - Monthly average
  - Weekly average
- 

### **Category Analyzer**

Computes

- Highest category
  - Lowest category
  - Category percentages
  - Top categories
- 

### **Trend Analyzer**

Computes

- Month-over-month growth
  - Week-over-week growth
  - Spending direction
- 

### **Habit Analyzer**

Computes

- Weekend spending
  - Night spending
  - Frequent merchants
  - Frequent categories
- 

### **Budget Analyzer**

Computes

- Budget usage
- Remaining budget
- Savings

- Burn rate

---

## Financial Health Score

Computes

One score

83 /100

Based on

- Savings
- Budget adherence
- Spending consistency
- Category diversification

No ML needed.

---

## Module 5 — Machine Learning Engine

This module contains ALL ML models.

Instead of one huge model

We build several.

---

### Model 1

Expense Categorization

#### Current

Input

Uber Ride



Category

Transport

Algorithm

**Random Forest**

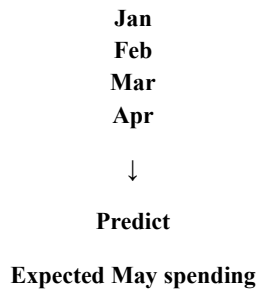
---

## Model 2

Budget Forecasting

Input

**Past expenses**



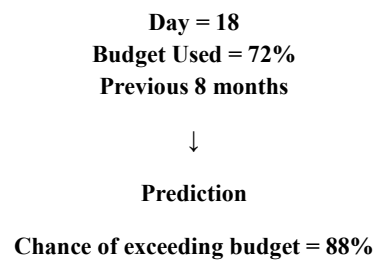
Possible algorithms

- Linear Regression (simple)
  - Random Forest Regressor
  - XGBoost Regressor
  - Prophet (time-series)
- 

## Model 3

Budget Risk Prediction

Input



Binary classification.

---

## Model 4

Anomaly Detection

Detect unusual expenses.



Example

Usually

Food = ₹400

Today

₹3500



Unusual spending detected.

Possible algorithms

- Isolation Forest
- Local Outlier Factor

---

## Model 5

Spending Pattern Clustering

Group users based on behavior.

Algorithms

### K-Means

Example

- Cluster **Saver**
- Cluster **Impulsive Spender**
- Cluster **Weekend Shopper**
- Cluster **Balanced User**

SIA can later use this.

---

## Module 6 — SIA

This is NOT an ML model.

This is an LLM orchestration module.

Responsibilities

- **Chat**
- **Voice**
- **Explain analytics**
- **Explain forecasts**
- **Generate insights**

- Answer questions

Input

**Financial Report**  
**ML Predictions**  
**User Question**

Output

**Natural language.**

Example

User - Why am I overspending?

SIA

Compared to last month, your food expenses increased by ₹3,200 and transport by ₹1,400. Weekend spending also rose by 22%, which is the primary reason your total expenses are higher.

Notice

**SIA didn't calculate. It explained.**

---

## Module 7 — Dashboard

Pure frontend.

---

## Module 8 — Notifications

Rules

Examples

**Budget 80%**



**Notify**

**Only ₹1,800 remaining.**

Later

## ML-generated notifications

Based on your spending habits, you're likely to exceed your entertainment budget this weekend.

---

## Module 9 — Settings

- Theme
- Currency
- Export
- Privacy
- Account

No ML.

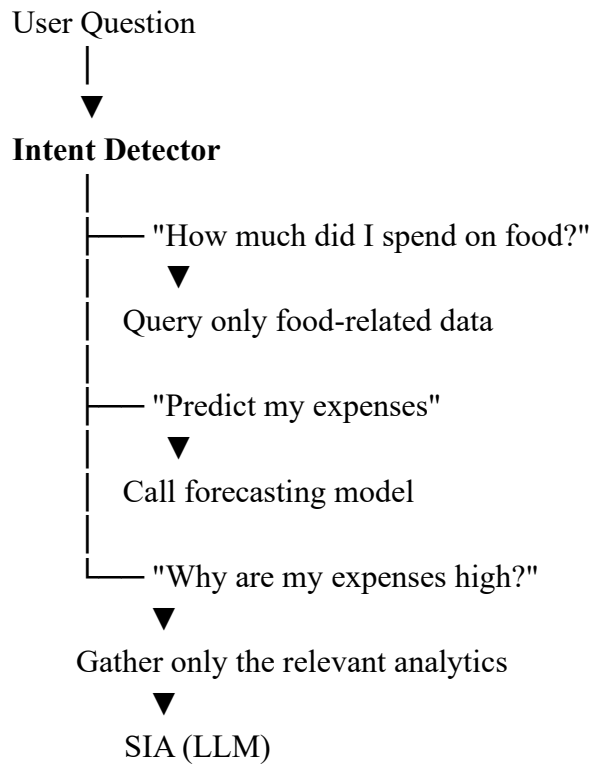
### Where ML is actually used

| Module         | ML?     | Purpose                                                |
|----------------|---------|--------------------------------------------------------|
| Authentication | ✗       | None                                                   |
| Expense CRUD   | ✓       | Expense categorization                                 |
| Budget         | ✓       | Budget risk prediction (future)                        |
| Analytics      | ✗       | Statistical calculations only                          |
| Dashboard      | ✗       | Visualization                                          |
| Notifications  | ✓       | Predictive alerts                                      |
| SIA            | ✗ (LLM) | Conversational reasoning over analytics and ML outputs |
| Forecasting    | ✓       | Month-end expense prediction                           |
| Reports        | ✗       | Aggregated financial summaries                         |

## A small architectural improvement

Avoid sending the **entire user report** to the LLM for every question. As the report grows, this becomes slower, more expensive, and can hit context limits.

Instead:



This "retrieve first, reason second" approach scales much better and mirrors how production AI assistants are typically built.

